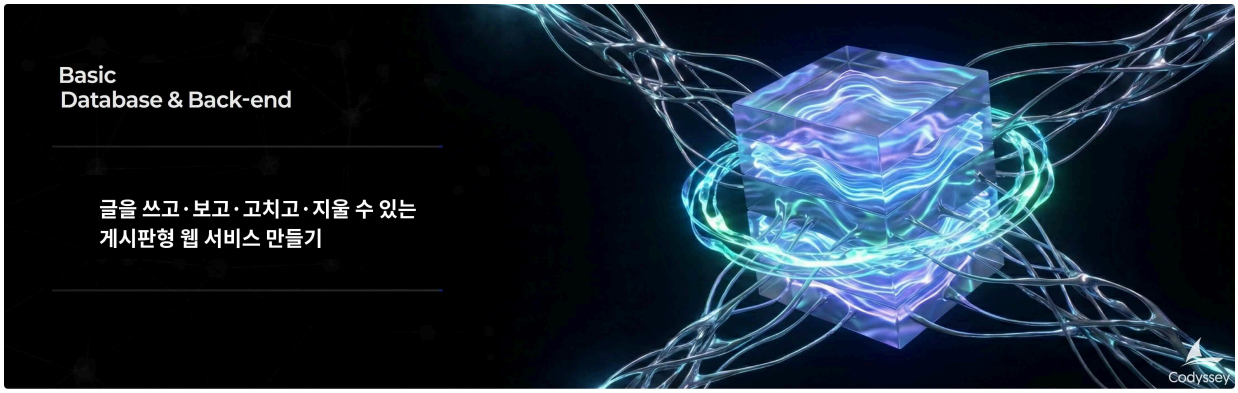
분야 AI/SW 기초 구분 데이터베이스와 백엔드 학습시간 60시간 필수/선택 선택

글을 쓰고·보고·고치고·지울 수 있는 게시판형 웹 서비스 만들기

문제기술

기술적 설명



1. 미션 소개

FastAPI가 빠르게 성장하는 이유는 단순히 빠르기 때문만이 아닙니다. 라우터, 서비스, 저장소와 같이 역할이 왜 나눠지는지 이해하면, 다른 어떤 프레임워크를 만나도 같은 사고로 접근할 수 있습니다. 브라우저 요청이 DB까지 갔다가 화면으로 돌아오는 전체 흐름을 직접 구현합니다.

FastAPI는 Python 기반 웹 애플리케이션을 개발할 때 빠르게 성장하고 있는 현대적 웹 프레임워크입니다. 요청 처리, 화면 렌더링, 데이터 저장 등 웹 서비스의 핵심 구성 요소를 직관적인 구조로 다룰 수 있도록 설계되어 있습니다.

이 미션에서는 FastAPI와 SQLAlchemy를 활용해 하나의 웹 애플리케이션을 처음부터 끝까지 직접 구현합니다. 단순 기능 구현이 아니라, 브라우저 요청이 서버 내부를 거쳐 데이터베이스에 저장되고 다시 화면으로 전달되는 전체 흐름을 경험합니다. 또한, 라우터, 서비스, 저장소로 역할을 분리하고, PRG 패턴과 ORM 기반 CRUD를 적용함으로써 웹 애플리케이션이 유지보수 가능하게 설계되는 이유와 구조적 장점을 이해하게 됩니다.

이 경험은 이후 인증/인가, 모델 간 연관관계 등 고급 주제로 확장되며, 다른 프레임워크(Django, Flask 등)를 학습할 때에도 동일한 구조적 사고(레이어 분리, 요청-응답 흐름 설계)를 적용할 수 있는 기반이 됩니다.

2. 최종 결과물

다음 기능이 정상 동작하는 FastAPI 웹 애플리케이션 1개를 완성한다.

1. **GET / (홈)**

- 브라우저에 "이 앱이 무엇을 하는지"를 설명하는 간단한 홈 화면이 출력된다.
- 홈 화면에는 아래 기능 화면으로 이동할 수 있는 링크가 존재한다.

2. 단일 도메인(예: Memo/Note/Task 등) CRUD 화면 흐름

- 사용자가 브라우저에서 데이터를 등록하고, 목록을 보고, 상세를 보고, 수정하고, 삭제할 수 있다.
- 화면은 Jinja2 템플릿 기반 SSR로 렌더링된다.

3. POST → Redirect → GET 흐름 적용

- 등록/수정/삭제 이후, 사용자가 새로고침해도 중복 요청이 발생하지 않도록 Redirect 흐름이 적용되어 있다.

4. 레이어 분리 구조

- 라우터(요청/응답), 서비스(비즈니스 로직), 저장소(데이터 접근)가 역할에 맞게 분리되어 있다.

5. SQLite + SQLAlchemy 연결 및 동작

- 인메모리 저장(리스트/딕셔너리)이 아니라 SQLite DB에 저장되는 CRUD가 동작한다.
- 단일 모델 1개(예: Memo) 기준으로 ORM 기반 저장/조회가 가능하다.

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- 브라우저 요청이 들어왔을 때 FastAPI 서버에서 "라우터 → 서비스 → 저장소 → 템플릿(뷰)"로 흐르는 과정을 말로 설명할 수 있다.
- GET 과 POST 의 역할 차이를 구분하고, 왜 분리해서 써야 하는지 설명할 수 있다.
- 폼 입력값이 서버로 전달되고, Pydantic 모델 또는 Form 파라미터로 다뤄지는 과정을 설명할 수 있다.
- PRG(Post-Redirect-Get) 패턴을 적용해야 하는 이유를 설명할 수 있다.
- SQLAlchemy ORM으로 단일 모델 CRUD가 동작하는 원리를 "내 코드 구조"를 근거로 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 서버 실행

- Python 가상환경을 생성하고, 필요한 패키지를 설치한다.
- 필수 패키지: `fastapi` , `uvicorn` , `sqlalchemy` , `jinja2` , `python-multipart`
- FastAPI 애플리케이션 실행 시 `http://localhost:8000` 에서 서버가 동작해야 한다.
- 터미널 로그에서 서버가 정상 기동되었음을 확인할 수 있어야 한다.

2. 프로젝트 구조

- 프로젝트 구조가 최소한 다음 역할을 분리한다.
    - `routers/` (요청/응답 처리 — FastAPI Router)
    - `services/` (비즈니스 로직)
    - `repositories/` (데이터 접근)
    - `models/` (SQLAlchemy ORM 모델 클래스)
- 템플릿 파일은 `templates/` 디렉터리 아래에 존재해야 한다.
- `requirements.txt` 또는 `pyproject.toml` 에 의존성 목록이 명시되어야 한다.

3. 화면 렌더링(SSR)

- 모든 주요 화면은 Jinja2 템플릿 렌더링 결과(`TemplateResponse`)로 출력되어야 한다.

4. 홈 화면

- URL: GET /
- 홈 화면에는 앱의 목적을 설명하는 문장 1~2줄이 포함되어야 한다.
- 홈 화면에는 최소 2개 이상의 링크(예: 목록, 등록 화면)가 존재해야 한다.

5. 도메인 CRUD 기능(단일 모델 1개)

- 학습자가 스스로 하나의 관리 대상을 정한다.
- "서비스 주제(데이터 종류)"는 학습자가 직접 정하되, 단일 모델 CRUD가 가능한 수준으로 제한한다. 아래는 참고할 수 있는 주제 및 필드 예시이다.
    - 메모 관리: 제목, 내용, 작성일시
    - 할 일 관리: 제목, 완료 여부, 마감일
    - 도서 관리: 제목, 저자, 출판년도
    - 일기장: 제목, 내용, 작성일, 기분
    - 단어장: 단어, 뜻, 예문

- **도메인 선택 가이드**: 모델의 필드는 3~6개 범위를 권장하며, 화면은 홈/목록/상세/등록(폼)/수정(폼) 최소 5개 흐름으로 구성한다.
- 아래 동작이 모두 가능해야 한다.
    - 목록 조회
    - 단건 조회(상세)
    - 등록
    - 수정
    - 삭제
- 상세 화면에는 해당 데이터의 전체 필드가 표시되어야 한다.\
- 수정 화면으로 이동하는 링크(또는 버튼), 삭제를 수행하는 링크(또는 버튼), 목록으로 돌아가는 링크가 존재해야 한다.
- 존재하지 않는 데이터 조회 시 적절한 안내 화면(예: "해당 데이터를 찾을 수 없습니다")이 표시되거나, 목록 화면으로 이동해야 한다.

6. 폼 기반 입력/수정 처리

- 등록/수정은 브라우저의 HTML Form을 통해 이루어진다.
- 등록/수정 폼에는 입력 필드(input/textarea 등)와 저장 버튼이 존재해야 하며, 목록으로 돌아갈 수 있는 취소 링크가 포함되어야 한다.
- 등록/수정 요청은 반드시 POST 방식으로 처리된다.
- 폼 데이터는 FastAPI의 `Form()` 파라미터로 수신한다.

7. Redirect 적용

- 등록/수정/삭제 이후에는 화면이 바로 렌더링되지 않고, 다른 화면으로 리다이렉트되어야 한다.
- `RedirectResponse` 를 사용하며, POST 이후 리다이렉트 시 `status_code=303` 을 적용한다.
- 새로고침(F5)으로 인해 같은 등록/수정/삭제가 중복 수행되지 않아야 한다.

8. 레이어 분리

- 라우터는 요청/응답과 화면 전환에 집중해야 한다.
- 비즈니스 로직은 서비스에 존재해야 한다.
- DB 접근은 저장소(Repository)에 존재해야 한다.

9. SQLite + SQLAlchemy 기반 저장

- 데이터는 서버 메모리(리스트/딕셔너리)만으로 관리하면 안 된다.

- SQLite DB 연결이 되어 있어야 하며, CRUD가 DB를 기준으로 동작해야 한다.
- SQLAlchemy ORM의 `Session` 을 사용하여 저장/조회가 가능해야 한다.
- DB 세션 관리는 FastAPI의 `Depends` 를 활용한 의존성 주입 방식을 사용한다.
- DB 파일(예: `database.db`)이 프로젝트 디렉터리에 생성되어야 하며, DB 브라우저(DB Browser for SQLite 등) 또는 Python 스크립트를 통해 저장된 데이터를 직접 확인할 수 있어야 한다.

10. **제출물 구성**
    * 전체 프로젝트 소스코드 (GitHub 저장소 링크 또는 압축 파일)
    * 실행 방법을 설명하는 README.md 파일 (가상환경 생성, 패키지 설치, 서버 실행 순서 포함)
    * (선택) 주요 화면 스크린샷

5. 보너스 과제 (선택)

1. 간단한 검색 기능 추가
    - 목록 화면에서 특정 조건(예: 제목 포함)으로 필터링할 수 있는 흐름을 추가해 본다.
    - 배움 포인트: 쿼리 파라미터를 활용한 GET 요청 처리와 SQLAlchemy의 `filter` / `like` 쿼리를 경험한다.
2. 간단한 검증 처리
    - 필수값이 비어 있으면 저장되지 않도록 하고, 화면에 안내 문구를 보여준다.
    - 배움 포인트: 서버 사이드 검증의 기본 흐름과 Pydantic validator 또는 수동 검증 방식을 경험한다.
3. 수정/삭제 UX 개선
    - 상세 화면에서 수정/삭제로 자연스럽게 이어지도록 링크/버튼을 설계해 본다.
    - 배움 포인트: 화면 간 네비게이션 흐름 설계와 사용자 경험 개선을 경험한다.

개발환경

6. 개발 환경

- Python 3.10 이상

제약조건

7. 제약 사항

- 의존성
  - fastapi , uvicorn , sqlalchemy , jinja2 , python-multipart 범위 내에서만 사용한다. 그 외 외부 라이브러리는 추가하지 않는다.
- 구조
  - 라우터, 서비스, 저장소, 모델 코드는 역할에 맞게 분리한다.
  - 템플릿 파일은 templates/ 디렉터리 아래에 둔다.
- 기능 범위
  - 로그인/권한/인증은 구현하지 않는다. 이 내용은 다음 미션에서 다룬다.
  - 모델 간 관계(연관관계)는 구현하지 않는다. 이 내용도 다음 미션에서 다룬다.
- 권장 사항
  - SQLAlchemy ORM 모델을 직접 라우터에서 반환하지 않고, Pydantic Schema(요청/응답 모델) 또는 별도 DTO 객체로 분리하여 사용하는 것을 권장

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구, 디자인은 얼마든지 달라도 된다.

- **홈 화면 예시 (GET /)**

```
나의 메모 앱

메모를 등록하고 목록/상세/수정/삭제할 수 있습니다.

[메모 목록 보기] [새 메모 작성]
```

- **목록 화면 예시**

```
메모 목록

- 오늘 배운 것 정리    [상세보기]
- 내일 할 일           [상세보기]

[새 메모 작성]
```